A

Project Report

on

Wi-Fi Dead Zone Mapper and Repeater

Submitted by,

Vardhit Pravin Ayachit- 2025BCS041

Harsh Mahesh Ambekar- 2025BCS099

Om Sachin Shendge- 2025BCS111

Niranjan Dinkar Satonkar- 2025BCS113

Under the Guidance of

Assistant Prof. Mr. Sachin Kumbhare

In Partial Fulfillment of Subject

MAKERSPACE (CEP-201)

(S.Y. B. Tech.) Sem III

Swami Ramanand Teerth Marathwada University,

Nanded (M.S.)



Department of Computer Science and Engineering,

Shri Guru Gobind Singhji Institute of Engineering and Technology (SGGSIE&T) Nanded MS INDIA

(2026 - 2027)

CERTIFICATE

This is to certify that, the Project titled ***"Wi-Fi Dead Zone Mapper and Repeater"*** which is being submitted here with for the subject Engineering Exploration of Shri Guru Gobind Singhji Institute of Engineering and Technology (SGGSIE&T) Nanded (An Autonomous Institute of Government of Maharashtra), affiliated to Swami Ramanand Teerth Marathwada University, Nanded (M.S. INDIA). This is the result of the work carried out by **Vardhit Ayachit- 2025BCS041, Harsh Ambekar- 2025BCS099, Om Shendge- 2025BCS111, Niranjan Satonkar- 2025BCS113** under my supervision and guidance.

This work embodied in this dissertation has not formed earlier for the basis of the award of any degree or compatible certificate or similar title of this for any other diploma/examining body or university to the best of knowledge and belief.

Place: Shri Guru Gobind Singhji Institute of Engineering and Technology, Nanded.
Date:

Assistant Prof. Mr. Sachin Kumbhare	**Dr. Milind V. Bhalerao**
The Guide	**The Dean**
Department of Mechanical Engineering	**(Innovation, Incubation and Linkages)**

DECLARATION

We, the undersigned students, hereby declare that the project work entitled ***"Wi-Fi Dead Zone Mapper and Repeater"*** which is being submitted to the Department of Electronics & Telecommunication Engineering of Shri Guru Gobind Singhji Institute of Engineering and Technology (SGGSIE&T), Nanded, in partial fulfillment of the requirements for the subject **Makerspace (CEP-201)** of B.Tech degree, is our original work carried out under the guidance of **Mr. Sachin Kumbhare**, Assistant Professor and **Dr. Milind V. Bhalerao, (Dean of Innovation, Incubation and Linkages).**

The work presented in this report has not been submitted earlier to any other university or institution for the award of any degree or diploma.

We further declare that all the information and facts presented in this report are true to the best of our knowledge and belief.

Place: Vishnupuri, Nanded MS INDIA

Date:

Signatures of Students:

1. **Vardhit Pravin Ayachit- 2025BCS041**
2. **Harsh Mahesh Ambekar- 2025BCS099**
3. **Om Sachin Shendge- 2025BCS111**
4. **Niranjan Dinkar Satonkar- 2025BCS113**

ACKNOWLEDGEMENT

The completion of our project work is a task which could not be accomplished without cooperation, motivation and help from our guide. At the outset, we would like to express our deep sense of gratitude to our guide, respected **Mr. Sachin Kumbhare**, for the valuable guidance and encouragement given consistently during the project work, without which it would not have been possible.

We are also very much grateful to the Lab In-charge of the Engineering Exploration Lab and Dean (Innovation, Incubation and Linkages), **Dr. Milind V. Bhalerao (The Dean of Innovation, Incubation and Linkages)**, who has been a constant source of inspiration.

We are very thankful to all the faculty members whose presence always inspires us to do better. Furthermore, we thank our friends for their support and the friendly environment. Finally, we would like to thank our families, particularly our parents, for their love, moral support and patience.

Vardhit Pravin Ayachit- 2025BCS041

Harsh Mahesh Ambekar- 2025BCS099

Om Sachin Shendge- 2025BCS111

Niranjan Dinkar Satonkar- 2025BCS113

ABSTRACT

Wi-Fi connectivity is not always consistent throughout a building: a usable connection near the router can become weak or unreliable in distant rooms, behind walls and in other obstructed areas, and the signal bars of a phone do not show where the problem lies. The project **Wi-Fi Dead Zone Mapper and Repeater** addresses this by measuring first, mapping the weak areas, placing a repeater strategically and verifying the result through another measurement session.

The software part is an Android application built on an open-source Wi-Fi analysis code base written in **Kotlin**. It is organised into three views: an **Access Point List** for inspecting nearby networks, a **Time Graph** for observing how a selected network's signal changes over time, and a floor-plan **Heatmap** in which the user uploads a floor plan, records the received signal strength (**RSSI**) at grid positions and generates a colour heat map by inverse-distance-weighted interpolation. The hardware part is a portable repeater: a **NodeMCU ESP8266** flashed with prebuilt repeater firmware, configured through its web page at **192.168.4.1** and powered from a **3.7 V 18650 Li-ion cell** through a 2-in-1 booster module.

The prototype demonstrates network inspection, firmware deployment, web-based configuration and a portable power arrangement. The application and the repeater are separate components linked by a user-driven workflow, and the improvement in coverage is to be established through a documented before-and-after comparison at the same measurement positions.

INDEX

Chapter No.	Topic Name		Page No.
	Certificate		i
	Declaration		ii
	Acknowledgement		iii
	Abstract		iv
	Table of Contents		v
	List of Symbols		viii
	List of Figures		ix
	List of Tables		x
	List of Abbreviations		xi
Chapter 1	**INTRODUCTION**		1
	1.1	Background of the Project	1
	1.2	Problem Statement	2
	1.3	Need of the Project	2
	1.4	Objectives of the Project	3
	1.5	Scope of the Project	3
	1.6	Methodology Overview	4
	1.7	Organization of the Report	4
Chapter 2	**LITERATURE SURVEY**		5
	2.1	History	5
	2.2	Existing Systems	5
		2.2.1 Technologies used in existing systems	7
	2.3	Comparative Study of Available Solutions	8
	2.4	Research Limitations of Existing Systems	9
		2.4.1 Limitations and drawbacks of each existing system	9
		2.4.2 Common problems observed	9
		2.4.3 Research gap	10
	2.5	Proposed Solution	10
Chapter 3	**SYSTEM DEVELOPMENT**		11
	3.1	Methodology	11

		3.1.1 Brief steps followed to complete the project	11
		3.1.2 High-level methodology	11
	3.2	System Block Diagram	12
		3.2.1 Explanation of each block and its function	12
		3.2.2 Interconnection between blocks	13
	3.3	Circuit Diagram	14
		3.3.1 Circuit Diagram	14
		3.3.2 Detailed working of the circuit	14
		3.3.3 Explanation of different sections	15
	3.4	Detailed Component Selection and Specifications	15
		3.4.1 List of major components with specifications	15
		3.4.2 Reason for selecting each component	17
		3.4.3 Technical parameters	17
	3.5	Software Design	19
		3.5.1 Algorithm	20
		3.5.2 Flowchart	22
Chapter 4	**IMPLEMENTATION**		23
	4.1	Hardware Implementation Details	23
		4.1.1 Step-by-step hardware assembly process	23
		4.1.2 Challenges faced during hardware building	23
	4.2	Programming Details	24
		4.2.1 Main program structure	24
		4.2.2 Important functions and their logic	25
		4.2.3 Code logic	25
	4.3	Interfacing of Sensors, Actuators, Modules	26
		4.3.1 Pin connections and interfacing diagram	26
		4.3.2 Communication protocols used	27
		4.3.3 How each module communicates with the microcontroller	27
	4.4	Development Tools Used	27
		4.4.1 List of software and hardware tools	27
		4.4.2 Purpose of each tool	27
Chapter 5	**PERFORMANCE ANALYSIS**		28
	5.1	Testing Methodology	28
		5.1.1 Types of testing done	28

		5.1.2 Test cases	28
		5.1.3 Testing conditions and environment	29
	5.2	Experimental Setup	30
		5.2.1 Description of the testing setup	30
		5.2.2 Photographs of experimental setup and working model	31
	5.3	Results	31
		5.3.1 Measured results in tabular form	31
		5.3.2 Graphs and charts	32
	5.4	Performance Analysis and Comparison with Existing Systems	34
		5.4.1 Analysis of obtained results	34
		5.4.2 Comparison table with existing systems	34
		5.4.3 Performance parameters	34
	5.5	Discussion on Results	35
		5.5.1 Interpretation of results	35
		5.5.2 Success points and reasons	35
		5.5.3 Failures or unexpected behavior and reasons	35
	5.6	Limitations	35
	5.7	Advantages	35
Chapter 6	**CONCLUSION AND FUTURE SCOPE**		36
	6.1	Conclusion	36
	6.2	Future Scope	36
	REFERENCES		37
	APPENDIX		38
	Appendix A: Bill of Materials		38
	Appendix B: Datasheets of major components		38
	Appendix C: Complete Source Code		38
	Appendix D: User Manual / Operating Instructions		40
	Appendix E: Additional Photographs		41

LIST OF SYMBOLS

Sr. No.	Symbols	Illustration
1	P_t	Transmit power (dBm)
2	G_t	Transmitter antenna gain (dBi)
3	G_r	Receiver antenna gain (dBi)
4	$PL(d)$	Path loss at distance d (dB)
5	$PL(d_0)$	Path loss at the reference distance (dB)
6	d	Distance between transmitter and receiver (m)
7	d_0	Reference distance (1 m)
8	n	Path-loss exponent
9	L_w	Attenuation of one wall (dB)
10	L_{sys}	System and body loss at the receiver (dB)
11	f	Carrier frequency (MHz)
12	w_i	IDW weight of reading i
13	d_i	Distance from a point to reading i (squares)
14	R	Influence radius of a reading (2.5 squares)
15	H	Heat-map colour hue (degrees)

LIST OF FIGURES

Figure No.	Illustration	Page No.
1.1	Formation of a Wi-Fi dead zone due to distance and wall attenuation	1
1.2	Methodology followed for the project	4
2.1	Typical views of a smartphone Wi-Fi analysis app (reference image, web article on Android Wi-Fi analyzer apps)	6
2.2	A widely shared DIY 'internet booster' circuit (not functional at 2.4 GHz)	7
2.3	Heat map with access points marked (reference image: Houston TechSys)	8
2.4	Office survey heat map with signal tooltip (reference image: "Calculating SSID Overhead")	8
3.1	Block diagram of the Wi-Fi Dead Zone Mapper and Repeater	12
3.2	Circuit and wiring diagram of the battery-powered repeater	14
3.3	Pin layout of the ESP8266 NodeMCU (CP2102) board	16
3.4	18650 Li-ion cell used	17
3.5	Power-bank module (T6845-C)	17
3.6	18650 battery holder (illustration)	17
3.7	Inverse-distance-weighted interpolation used for the heat map	21
3.8	Flowcharts of (a) the heat-map survey and (b) the repeater start-up	22
4.1	Software architecture of the Heatmap module	25
5.1	Department floor plans prepared as survey maps	30
5.2	Planned survey grid, checkpoints and proposed repeater position	31
5.3	NodeMCU connected over USB	31
5.4	Colour scale used by the heat map of the application	33
E.1	Underside of the power-bank module (controller IC and boost inductor)	43
E.2	Typical single-cell 18650 boost arrangement (reference image: LEAP#501)	43

LIST OF TABLES

Table No.	Title of the Table	Page No.
2.1	Milestones in Wi-Fi and coverage-extension technology	5
2.2	Comparative study of available solutions	9
3.1	Interconnection between blocks and signal types	13
3.2	Major components with specifications	15
3.3	Radio, electrical and firmware parameters of the ESP8266 repeater	18
3.4	Estimated coverage radius of the ESP8266 repeater	18
4.1	Files of the Heatmap module and their responsibilities	24
4.2	Pin connections of the repeater	26
5.1	Unit tests of the Heatmap module (HeatmapIdwTest)	29
5.2	Test plan of the prototype	29
5.3	Results demonstrated by the current prototype	32
5.4	Comparison of the proposed system with existing systems	34
A.1	Bill of materials of the repeater	38
F.1	Measurement record sheet for the before-and-after comparison	**Error! Bookmark not defined.**

LIST OF ABBREVIATIONS

Sr. No.	Abbreviations	Illustration
1	AP	Access Point
2	API	Application Programming Interface
3	dBi	Decibels relative to an isotropic antenna
4	dBm	Decibels relative to one milliwatt
5	DHCP	Dynamic Host Configuration Protocol
6	DNS	Domain Name System
7	GPIO	General Purpose Input/Output
8	HTTP	Hypertext Transfer Protocol
9	IDE	Integrated Development Environment
10	IDW	Inverse Distance Weighting
11	IEEE	Institute of Electrical and Electronics Engineers
12	ISM	Industrial, Scientific and Medical
13	JSON	JavaScript Object Notation
14	LDO	Low-Dropout Regulator
15	LED	Light Emitting Diode
16	Li-ion	Lithium-ion
17	lwIP	Lightweight Internet Protocol stack
18	NAPT	Network Address and Port Translation
19	NAT	Network Address Translation
20	PNG	Portable Network Graphics
21	RSSI	Received Signal Strength Indicator
22	SDK	Software Development Kit
23	SoC	System on Chip
24	SSID	Service Set Identifier
25	STA	Station (Wi-Fi client mode)
26	UART	Universal Asynchronous Receiver-Transmitter
27	USB	Universal Serial Bus
28	WPA2	Wi-Fi Protected Access 2

Chapter 1

INTRODUCTION

1.1 Background of the Project

Wi-Fi connectivity is not always consistent throughout a building: a user may have a usable connection near the router but weak or unreliable connectivity in distant rooms, behind walls or in other obstructed areas. Coverage varies from one location to another, which makes it difficult to identify exactly where the problem occurs [1].

The 2.4 GHz radio signal does not spread uniformly. Its power falls with the square of distance even in free space, and indoors walls, furniture and people absorb and reflect it further [2]. The received signal strength indicator (RSSI), reported in dBm by every Wi-Fi device, measures this: better than about −60 dBm gives fast, stable links, while below about −80 dBm connections usually drop.

Areas where the signal is too weak for reliable communication are called Wi-Fi dead zones (Figure 1.1); this project focuses on identifying and visualising them before attempting to improve coverage.

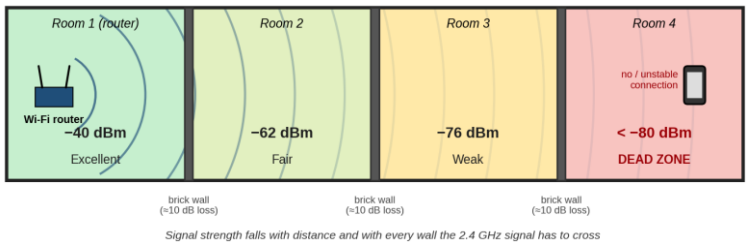


FIGURE 1.1 – FORMATION OF A WI-FI DEAD ZONE DUE TO DISTANCE AND WALL ATTENUATION

Real-Life Applications:

1. *Online classes and work from home:* Classes and video meetings are interrupted in rooms far from the router.
2. *Large homes, hostels and student accommodation:* Walls, router placement and room layouts create uneven coverage.

3. *Small offices, laboratories and smart devices:* Devices at the far end of a room or building may receive weaker Wi-Fi or fail to stay connected.

1.2 Problem Statement

- *Signal bars only:* Users rely on the Wi-Fi icon or signal bars, which do not give a room-by-room coverage map.
- *Late discovery:* A weak connection is often noticed only after a video call, online class or download starts failing.
- *Unguided fixes:* Moving a router or buying an extender without identifying the weak area may not solve the actual problem.
- *Lists are not maps:* A list of nearby networks is useful, but it does not show how coverage varies across a floor plan.

The problem addressed is therefore: how can Wi-Fi coverage be examined at different locations, represented visually on a floor plan and used to guide the placement of a repeater?

1.3 Need of the Project

A network may work properly in one room and become unreliable in another; instead of guessing, users can compare measurements from different locations on a floor plan.

Why the application is useful:

- *No extra instrument:* It uses the Wi-Fi receiver already present in an Android phone.
- *Measurement instead of guesswork:* Weak areas and candidate repeater positions are decided from recorded readings.
- *Easy to read and repeat:* A colour map on the floor plan is easy to understand, and saved surveys allow a before-and-after comparison.

Users and target audience:

- *Home users:* Identify rooms with consistently weak Wi-Fi, check whether the router location causes uneven coverage and plan a better repeater position.
- *Students and hostel residents:* Investigate weak coverage around study areas, bedrooms and shared spaces, and learn wireless networking hands-on.

- *Small offices and educational laboratories:* Examine coverage across workstations, meeting rooms and laboratory spaces to guide further testing.
- *Networking learners and project teams:* Explore Wi-Fi scanning, signal visualisation, embedded hardware and repeater configuration.

How it can be used:

A user notices that Wi-Fi works near the living-room router but is unreliable at a study desk in a distant bedroom. The user records observations at several positions, marks the weak region on the floor plan, tests a repeater placement intended to improve connectivity there and measures again to confirm the result.

1.4 Objectives of the Project

Main Objective:

To develop a measurement-driven approach to Wi-Fi coverage in which weak areas are first measured and mapped on a floor plan, a portable ESP8266 repeater is then placed strategically, and the result is verified through another measurement session.

Specific Objectives:

- To display information about nearby Wi-Fi access points.
- To observe how the signal from a selected access point changes over time.
- To use a floor plan to associate Wi-Fi observations with physical locations.
- To visualise weak and stronger coverage areas using a heat map and identify candidate repeater locations.
- To build and demonstrate a portable ESP8266 repeater prototype.
- To establish a practical workflow for comparing coverage before and after a network change.

1.5 Scope of the Project

Features and Limits:

- *Wi-Fi analysis application:* Three main views on an Android phone: Access Point List, Time Graph and floor-plan Heatmap.

- *Repeater prototype:* A NodeMCU ESP8266 with prebuilt repeater firmware, configured at 192.168.4.1 and powered from an 18650 cell through a booster module.
- *User-driven workflow:* The user measures, maps the weak areas, places the repeater and measures again.

What is not included:

- *Automatic integration:* The application and the repeater are separate components and do not communicate with each other.
- *Automatic placement:* The heat map guides the placement decision but does not compute an optimal position.
- *Higher bands:* The ESP8266 works only in the 2.4 GHz band.

1.6 Methodology Overview

The core idea of the project is to measure first, map the weak areas, place the repeater strategically and verify the result through another measurement session. The seven steps followed are shown in Figure 1.2.

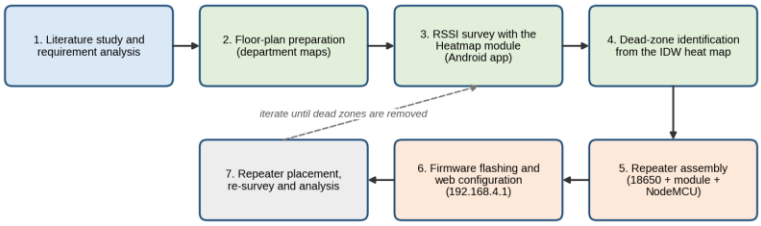


FIGURE 1.2 – METHODOLOGY FOLLOWED FOR THE PROJECT

1.7 Organization of the Report

Chapter 1 introduces the project; Chapter 2 surveys the literature and existing systems; Chapter 3 explains the system design, circuit, components and software; Chapter 4 describes the implementation; Chapter 5 presents the test plan, validation and results; and Chapter 6 gives the conclusion and future scope.

Chapter 2

LITERATURE SURVEY

2.1 History

Wireless networking began with ALOHAnet at the University of Hawaii in 1971, which first demonstrated packet radio on a shared channel. In 1985 the U.S. Federal Communications Commission opened the industrial, scientific and medical (ISM) bands, including 2.4 GHz, for unlicensed spread-spectrum use, which made low-cost wireless LAN products possible. The first IEEE 802.11 standard (1997) offered 2 Mbps; later amendments, from 802.11b (1999) to 802.11ax (Wi-Fi 6), raised speed and efficiency [1].

As speeds increased, coverage became the main complaint of users. Repeaters evolved from the wireless distribution system (WDS) to plug-in range extenders and, from about 2016, to mesh systems. On the measurement side, the RADAR system showed that useful signal-strength maps can be built from ordinary Wi-Fi readings [3], and the low-cost ESP8266 Wi-Fi chip released in 2014 made complete Wi-Fi devices possible for a few hundred rupees [4]. The milestones are summarised in Table 2.1.

TABLE 2.1 – MILESTONES IN WI-FI AND COVERAGE-EXTENSION TECHNOLOGY

Year	Milestone	Significance
1971	ALOHAnet	First packet-radio network
1997	IEEE 802.11 (2 Mbps)	First wireless LAN standard
1999	802.11b / 802.11a, Wi-Fi Alliance	11 Mbps and 54 Mbps; interoperability
2014	ESP8266 Wi-Fi SoC	Wi-Fi devices at very low cost
2019–2021	802.11ax (Wi-Fi 6)	Higher efficiency in dense networks

2.2 Existing Systems

1. *Plug-in range extenders:* Single-radio devices that receive and retransmit the router's signal under a new name; about ₹1,200–₹3,000, and they need a mains socket.
2. *Mesh Wi-Fi systems:* Several nodes forming one network with automatic roaming; best coverage, but ₹8,000 or more for two nodes.

3. *Powerline adapters with Wi-Fi:* Carry data over the electrical wiring to a new access point; performance depends on the wiring.
4. *Site-survey software:* Records signal strength over a floor plan to produce heat maps; full versions need paid licences and training.
5. *Smartphone Wi-Fi analysis apps:* Show RSSI, channels and nearby networks; they measure the problem but do not map or solve it.

Typical analysis apps show a list of visible networks, a graph of the channels in use and a signal–time graph (Figure 2.1); they describe the radio environment at one location but do not by themselves map a whole floor.

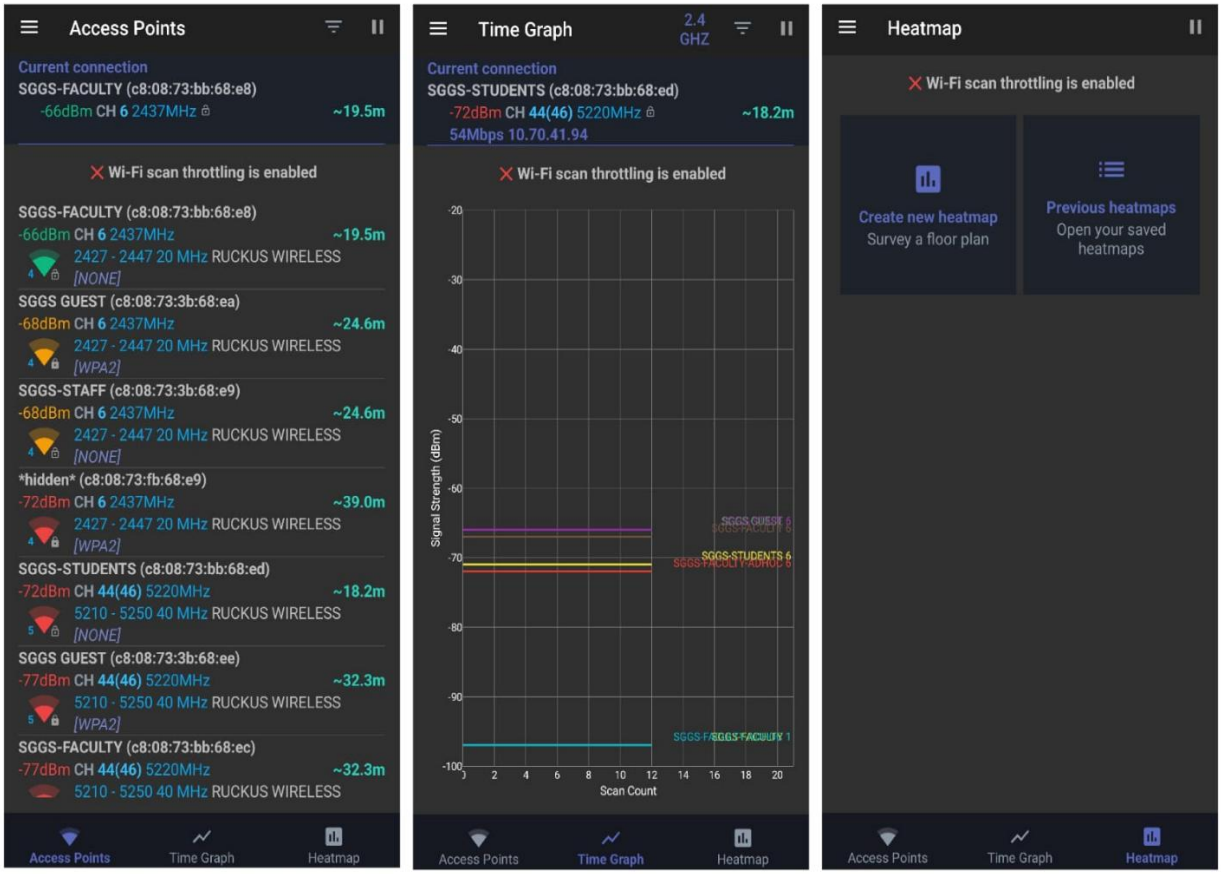


FIGURE 2.1 – TYPICAL VIEWS OF A SMARTPHONE WI-FI ANALYSIS APP (REFERENCE IMAGE, WEB ARTICLE ON ANDROID WI-FI ANALYZER APPS)

In addition, low-cost 'internet booster' circuits are widely shared on social media. A typical example, shown in Figure 2.2, connects an LM386 amplifier, a 27 MHz crystal, coil antennas and an 18650 cell and claims to boost the internet signal.



FIGURE 2.2 – A WIDELY SHARED DIY 'INTERNET BOOSTER' CIRCUIT (NOT FUNCTIONAL AT 2.4 GHZ)

Such a circuit cannot amplify Wi-Fi: the LM386 is an audio amplifier, the 27 MHz crystal is about ninety times below the 2.4 GHz band, and there is no receiver. A working extender must receive, decode and retransmit 802.11 frames, as a Wi-Fi chip such as the ESP8266 does.

2.2.1 Technologies used in existing systems

How a commercial Wi-Fi repeater works:

A Wi-Fi repeater receives a wireless network signal and retransmits it so that connectivity reaches areas beyond the original router's useful coverage. A typical plug-in repeater is used as follows:

1. *Placement:* It is plugged in roughly halfway between the router and the weak area, where it still receives a usable signal; many models show a signal LED or app indicator for this.
2. *Pairing:* It is linked to the router with the WPS button or a set-up page or app, and copies the router's network name (often with an _EXT suffix) and password.
3. *Relaying:* It receives data from the router over Wi-Fi and retransmits it to devices in the extended area, and their replies back to the router.
4. *Radio use:* A single-band model uses one radio and channel for both links, roughly halving throughput; a dual-band model can use one band towards the router and the other towards the clients.
5. *Variants:* Models with an Ethernet port can act as wired access points, and mesh systems use several coordinated nodes with one network name and automatic hand-over.

A router manages network traffic and provides the original network, whereas a repeater only extends the reach of an existing network: it does not increase the internet speed supplied by the provider. Other technologies used in existing systems are:

- *Routing with NAT:* ESP8266-based repeaters create a separate subnet and translate addresses using NAPT [5].
- *Heat mapping by interpolation:* Survey tools fill the gaps between measured points by spatial interpolation, commonly inverse-distance weighting (IDW), in which nearer readings carry more weight [6]; predictive tools use indoor path-loss models [7].

Professional site-survey tools present the result as a colour heat map over the floor plan, often with the access points marked and the measured level shown on demand (Figure 2.3 and Figure 2.4).

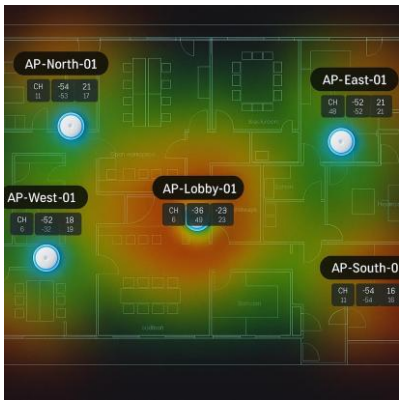


FIGURE 2.3 – HEAT MAP WITH ACCESS POINTS MARKED (REFERENCE IMAGE: HOUSTON TECHSYS)

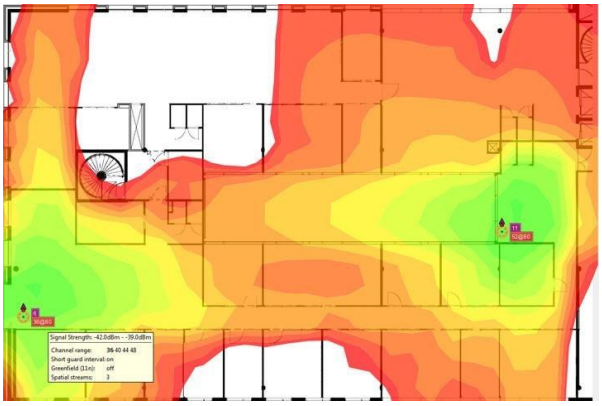


FIGURE 2.4 – OFFICE SURVEY HEAT MAP WITH SIGNAL TOOLTIP (REFERENCE IMAGE: "CALCULATING SSID OVERHEAD")

2.3 Comparative Study of Available Solutions

Table 2.2 compares the existing solutions with the proposed system.

TABLE 2.2 – COMPARATIVE STUDY OF AVAILABLE SOLUTIONS

Parameter	Range extender	Mesh Wi-Fi system	Site-survey software	Proposed system
Cost	₹1,200–₹3,000	₹8,000 and above	Licence-based	Under ₹1,000
Coverage gain	Good (one node)	Excellent	None (measures only)	Good (one portable node)
Throughput	About half of the link	High	Not applicable	Reduced (relayed by ESP8266)
Power	Mains socket	Mains socket	Laptop / phone	18650 battery
Dead-zone mapping	No	Limited	Yes, detailed	Floor-plan heat map on the phone
Advantages	Simple set-up	Seamless roaming	Accurate analysis	Low cost, portable, map-guided
Disadvantages	Placement by guesswork	High cost	No repair; costly	Low throughput; 2.4 GHz only

2.4 Research Limitations of Existing Systems

2.4.1 Limitations and drawbacks of each existing system

- *Range extenders:* Placed by guesswork; inside the weak area they repeat an already poor signal, and the mains plug limits placement.
- *Mesh systems:* Too costly for most students and departments, and every node needs a socket.
- *DIY 'booster' circuits:* As shown in Section 2.2, they work at the wrong frequency and cannot process Wi-Fi.

2.4.2 Common problems observed

- *Diagnosis separated from cure:* Tools that map the signal do not extend it, and devices that extend it do not show where they are needed.
- *Dependence on mains power:* Almost every solution requires a power socket at the installation point.
- *Cost:* Effective solutions are priced beyond the budget of typical student users.

2.4.3 Research gap

There is no low-cost, portable solution that combines a floor-plan signal survey with a repeater placed exactly where the survey indicates. Existing systems either measure coverage without improving it or improve it without measuring it. An Android phone for mapping and a battery-powered ESP8266 repeater can fill this gap at a fraction of the cost of commercial equipment.

2.5 Proposed Solution

The proposed system combines two complementary components. The first is the Wi-Fi Deadzone Mapper Android application, built on an open-source Wi-Fi analysis code base [8]: it provides an Access Point List, a Time Graph and a floor-plan Heatmap in which the user uploads a floor plan, collects measurements at different positions and represents them as a heat map [9]. The second is a NodeMCU ESP8266 configured with prebuilt repeater firmware [10] and powered by an 18650 cell through a voltage-boosting module, so that the repeater can be placed where the map suggests.

Key Improvements over Existing Systems:

- *Map-guided placement:* The repeater is placed where the heat map shows a good main signal at the edge of the dead zone.
- *Battery operation:* Portable power from one 18650 cell, without a power socket.
- *Very low cost:* Under ₹1,000 (approximately) for the complete repeater.
- *Measure, place, verify:* The repeater position is checked by repeating the same measurements.

Chapter 3

SYSTEM DEVELOPMENT

3.1 Methodology

The mapping stage shows how the main network's signal is distributed over the floor plan; the repeater stage then adds a second transmitter at a point chosen from that map.

3.1.1 Brief steps followed to complete the project

- *Requirements and maps:* Defined the dead-zone threshold (below −80 dBm) and prepared floor-plan images of the CSE, ENTC and Instrumentation departments.
- *Application:* Used the open-source analyzer as the base, with the Access Point List and Time Graph, and added the floor-plan Heatmap module.
- *Initial repeater prototype:* Flashed prebuilt repeater firmware to the NodeMCU, powered it from a laptop and configured it through its web page.
- *Portable power and verification:* Assembled the 18650 cell, holder and booster module, and planned the before-and-after measurement of coverage.

3.1.2 High-level methodology

Hardware approach: The repeater is a self-contained node: the 18650 cell stores 7.4 Wh, the 2-in-1 module boosts its 3.0–4.2 V to 5 V, and the NodeMCU regulates this to 3.3 V for the ESP8266. During development the board was first powered directly from a laptop over USB.

Software approach: The Android application, written in Kotlin, scans Wi-Fi networks periodically; the team's Heatmap module uses the connected access point's signal level to build a floor-plan survey and an IDW heat map. The repeater runs pre-built open-source C firmware that operates the ESP8266 as station and access point at once and routes packets between them with network address and port translation (NAPT).

From heat map to repeater placement:

1. *Identify the weak region:* Find the floor-plan locations where the measured signal is weaker than at the other tested locations.

2. *Examine the surrounding area:* Find a nearby position where the router's signal is still usable, because the repeater must receive the network before retransmitting it.
3. *Select a candidate location:* Choose a position that keeps a reasonable link to the router while extending coverage towards the weak region.
4. *Configure the repeater:* Set up the ESP8266 through its configuration page with the required network details.
5. *Measure again:* Repeat the observations at the original positions and compare the readings and heat maps.

The heat map guides the placement decision; it does not determine an optimal position automatically or guarantee an improvement, so the final position must be verified experimentally.

3.2 System Block Diagram

The architecture of the system is shown in Figure 3.1. It consists of a software subsystem (the Android application) and a hardware subsystem (the battery-powered ESP8266 repeater). The two are separate functional components: the user links them by measuring, deciding the placement and measuring again, and no automatic communication takes place between them.

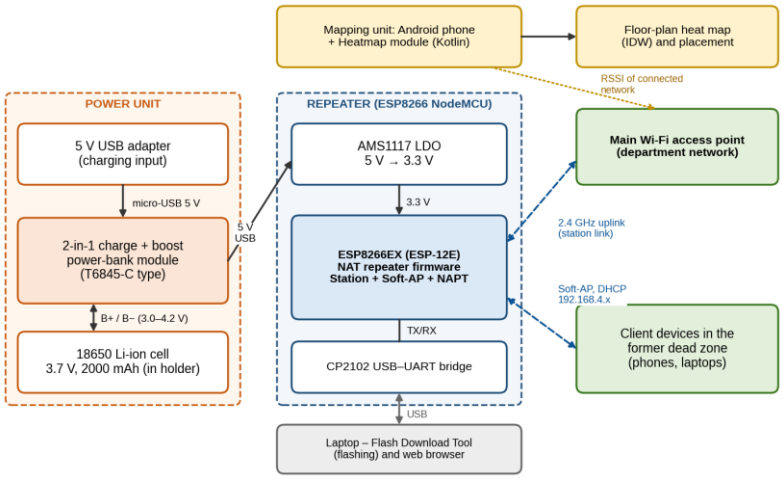


FIGURE 3.1 – BLOCK DIAGRAM OF THE WI-FI DEAD ZONE MAPPER AND REPEATER

3.2.1 Explanation of each block and its function

Software subsystem – Android application:

- Scans and displays the available access points (Access Point List).
- Shows the signal strength of a selected access point over time (Time Graph).

- Provides the floor-plan heat-map workflow for spatial coverage analysis (Heatmap), helping the user find locations that need further investigation.

Hardware subsystem – ESP8266 repeater:

- *Power unit:* The 18650 cell in its holder feeds the 2-in-1 module, which boosts it to 5 V at the USB-A output.
- *Controller:* The NodeMCU's AMS1117 supplies 3.3 V to the ESP8266EX, which runs the prebuilt repeater firmware; the CP2102 serves flashing and the console.
- *Radio links and configuration:* The station interface joins the main access point, the soft-AP gives clients 192.168.4.x addresses and relays their traffic through NAPT, and settings are entered on the device-hosted page at 192.168.4.1.

3.2.2 Interconnection between blocks

Table 3.1 summarises the connections between the blocks.

TABLE 3.1 – INTERCONNECTION BETWEEN BLOCKS AND SIGNAL TYPES

Interface link	Blocks connected	Signal type
B+/B− wires, USB cable	Cell → module → NodeMCU	DC 3.0–4.2 V, then 5 V
Station / soft-AP links	NodeMCU ↔ access point / clients	2.4 GHz Wi-Fi, DHCP 192.168.4.x
Survey measurement	Access point → smartphone	RSSI of the connected network
Configuration	Browser / laptop ↔ repeater	HTTP 192.168.4.1; USB–UART 115200 baud

User-driven integration workflow:

1. Inspect the available Wi-Fi networks and observe the selected network's signal over time.
2. Take measurements at different floor-plan positions and identify the weak-coverage region.
3. Select a repeater location where the original network is still usable, then repeat the measurements to see whether coverage changed.

3.3 Circuit Diagram

3.3.1 Circuit Diagram

The wiring is shown in Figure 3.2: the holder's red lead goes to B+ and its black lead to B− of the module, and a short USB-A to micro-USB cable carries 5 V to the NodeMCU. No other parts are needed; the status indicator is the ESP-12E's blue LED, driven by the firmware on GPIO2.

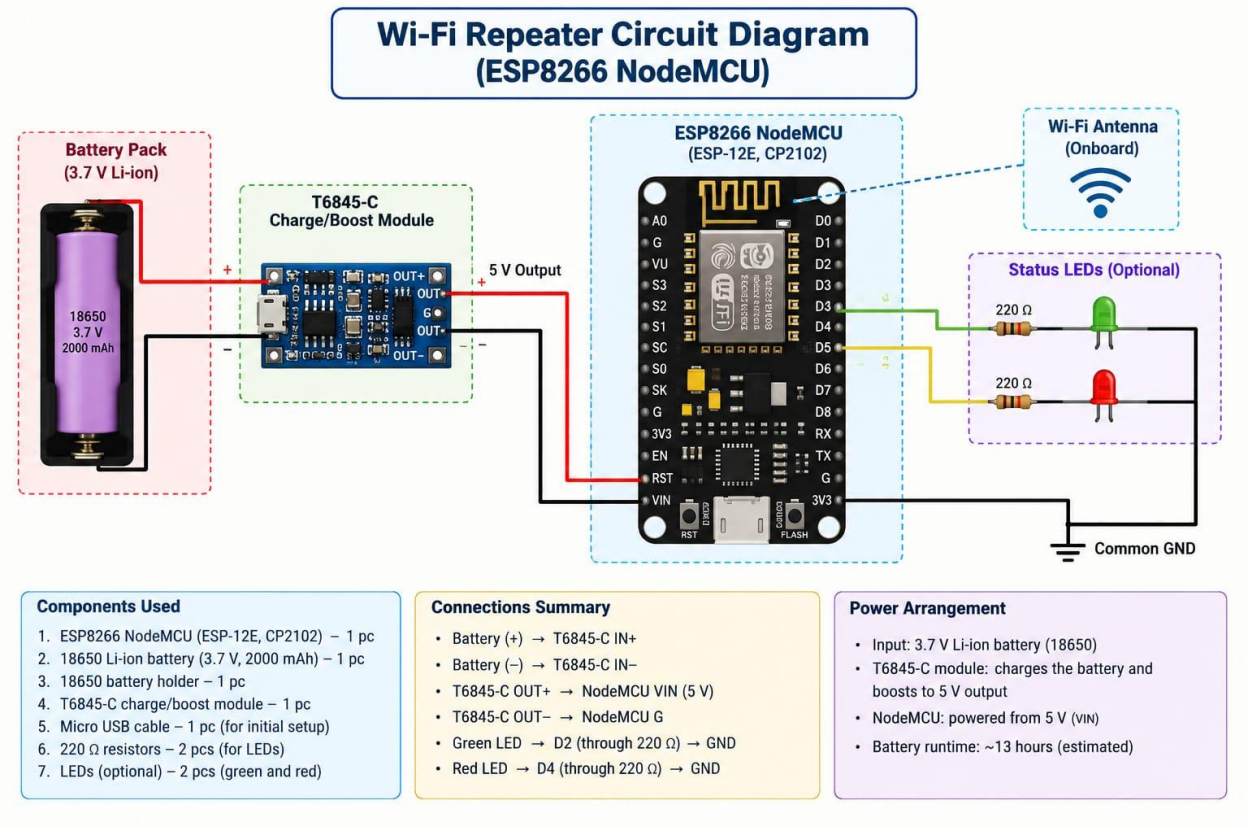


FIGURE 3.2 – CIRCUIT AND WIRING DIAGRAM OF THE BATTERY-POWERED REPEATER

3.3.2 Detailed working of the circuit

1. *Charging (as specified):* According to its published specifications the module charges the cell from a 5 V micro-USB input at up to 1 A to 4.2 V [11]; this function was not separately verified in the prototype.
2. *Boosting:* The boost converter raises the cell voltage (3.0–4.2 V) to a regulated 5 V, about 85 % efficient at full load [11].

3. *Regulation and radio:* The AMS1117 low-dropout (LDO) regulator gives 3.3 V; the firmware joins the main access point and forwards packets (about 75–90 mA at 5 V), with the LED flashing once per second when connected [10].
4. *Protection:* At about 2.9 V the module switches off its output; it also protects against short circuits and over-temperature.

3.3.3 Explanation of different sections

- *Power, control and RF sections:* The cell and module supply 5 V, the AMS1117 gives 3.3 V and the ESP-12E performs all Wi-Fi and routing functions.
- *Indication and programming section:* The LED on GPIO2 shows the state; the CP2102 serves flashing and the 115200-baud console.

3.4 Detailed Component Selection and Specifications

The components were selected for low cost, local availability, adequate radio performance and safe battery operation.

3.4.1 List of major components with specifications

Table 3.2 lists the major components of the system; each one is described below.

TABLE 3.2 – MAJOR COMPONENTS WITH SPECIFICATIONS

Component	Model / Type	Key specifications	Figure
Wi-Fi controller	ESP8266 NodeMCU (ESP-12E, CP2102)	80/160 MHz, 802.11 b/g/n, 4 MB flash	Fig. 3.3
Battery	18650 Li-ion cell	3.7 V, 2000 mAh, 7.4 Wh	Fig. 3.4
Power module	2-in-1 power-bank module (T6845-C)	1 A charging, 5 V / 1 A boost output	Fig. 3.5
Battery holder	Single-cell 18650 holder	Spring contacts, red/black leads	Fig. 3.6
Mapping device	Android smartphone	Android 7.0 or later, Wi-Fi	—

ESP8266 NodeMCU (CP2102) Board

The ESP8266EX from Espressif Systems is a low-cost Wi-Fi system-on-chip that combines a 32-bit Tensilica L106 processor (80 MHz, up to 160 MHz), a complete 2.4 GHz 802.11 b/g/n radio and a TCP/IP stack, so one chip can join a Wi-Fi network without a separate microcontroller [4].

It can work as a station (client), as a soft access point or as both at once, which is what makes a repeater possible. The ESP-12E module adds 4 MB of flash memory, a 26 MHz crystal and a printed antenna, and the NodeMCU board adds a CP2102 USB-to-UART bridge, an AMS1117 3.3 V regulator, FLASH and RST buttons, 11 digital GPIO pins and one analogue input, so it is powered and programmed over one USB cable. Its pin layout is shown in Figure 3.3 and the board used is photographed in Figure 5.3. Although the shield is marked 'PA +25 dBm', the datasheet maximum of +20 dBm (802.11b) is used in this report.

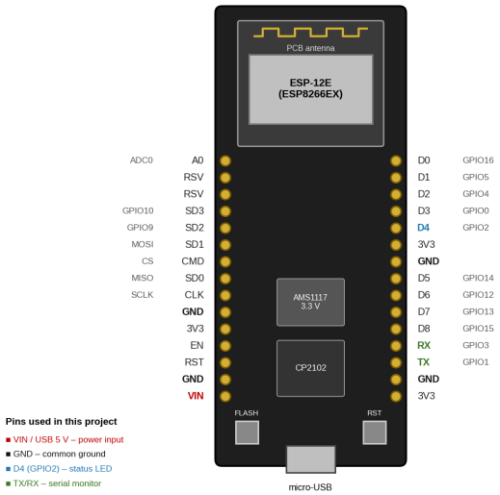


FIGURE 3.3 – PIN LAYOUT OF THE ESP8266 NODEMCU (CP2102) BOARD

18650 Li-ion Cell

A cylindrical 18650 lithium-ion cell (18 mm diameter, 65 mm length) stores the energy for portable operation (Figure 3.4). The cell used is rated 3.7 V nominal, 2000 mAh and 7.4 Wh, and its label states compliance with IS 16046 / IEC 62133, the safety standard for portable lithium cells. It is charged to 4.2 V and should not be discharged below about 2.9–3.0 V.

2-in-1 Power-Bank Module (T6845-C)

This 21.7 mm × 17.2 mm 2-in-1 discharge/booster module (Figure 3.5) has a micro-USB input, a USB-A output and B+/B− pads for the cell. Its published specifications list charging at up to 1 A to 4.2 V, a 5 V output of up to 1 A, a cut-off at about 2.9 V and about 85 % efficiency [11]; the output, current capacity, protection and charging behaviour should be verified on the actual module before relying on them.

18650 Single-Cell Battery Holder

The holder (Figure 3.6) grips the cell between a spring (−) and a flat (+) contact and brings out red and black leads, so the cell can be replaced without desoldering.



FIGURE 3.4 – 18650 LI-ION CELL USED



FIGURE 3.5 – POWER-BANK MODULE (T6845-C)

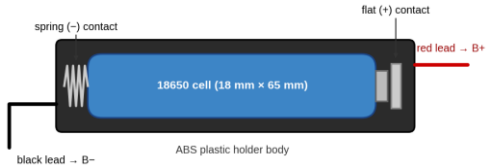


FIGURE 3.6 – 18650 BATTERY HOLDER (ILLUSTRATION)

Android Smartphone (Mapping Device)

Any Android phone with Android 7.0 or later can run the application. During a survey the phone must stay connected to the network being surveyed, because the Heatmap module records the signal level of the connected access point.

3.4.2 Reason for selecting each component

- *ESP8266 NodeMCU:* A very cheap board with a complete Wi-Fi radio, station + access-point operation and USB flashing.
- *18650 cell, holder and module:* 7.4 Wh in about 45 g, replaceable without soldering, with a regulated 5 V output.
- *Android smartphone:* Already available; its Wi-Fi receiver reports RSSI with no extra hardware.

3.4.3 Technical parameters

The radio, electrical and firmware parameters that decide the performance of the repeater are listed in Table 3.3.

TABLE 3.3 – RADIO, ELECTRICAL AND FIRMWARE PARAMETERS OF THE ESP8266 REPEATER

Parameter	Value	Remarks
Wi-Fi standard	802.11 b/g/n, 2.4 GHz	Soft-AP limited to 802.11 b/g [12]
Transmit power	+20 dBm (b), +17 dBm (g), +14 dBm (n)	As per ESP8266EX datasheet
Average current	About 80 mA (station + soft-AP)	Peaks of about 170 mA while transmitting
Soft-AP clients	Up to 8 stations	Firmware setting max_clients [10]

Path-loss model: The loss of the 2.4 GHz signal over a distance d is estimated with the log-distance model including wall losses [2], [7]:

$$PL(d) = PL(d_0) + 10\, n \log_{10}(d/d_0) + \Sigma L_w \tag{3.1}$$

where $PL(d_0)$ = 40.2 dB at d_0 = 1 m (2437 MHz), n is the path-loss exponent and ΣL_w the loss of the walls crossed; the received signal strength is then:

$$\text{RSSI} = P_t + G_t + G_r - PL(d) - L_{sys} \tag{3.2}$$

Range of the repeater: With Equations (3.1) and (3.2), a +17 dBm soft-AP output, a 2 dBi antenna, a 6 dB system loss and about 6 dB per brick wall, the distances for a good (−67 dBm) or usable (−75 dBm) signal are estimated in Table 3.4 (calculations, not measurements). As the repeater must sit where the router is still usable, indoors it extends coverage by one or two rooms, for up to eight clients sharing about 5 Mbps [10].

TABLE 3.4 – ESTIMATED COVERAGE RADIUS OF THE ESP8266 REPEATER

Environment	Good signal (−67 dBm)	Usable signal (−75 dBm)
Open line of sight (n = 2)	about 100 m	about 245 m
Indoors, same room or corridor (n = 3)	about 21 m	about 39 m
Indoors, through one brick wall	about 13 m	about 25 m
Indoors, through two brick walls	about 8 m	about 16 m

Signal-quality classification: RSSI values are grouped as excellent (−50 dBm or better), good (−50 to −60), fair (−60 to −70), weak (−70 to −80) and dead zone (below −80 dBm); the corresponding

heat-map colours are shown in Section 5.3.2. While surveying, the status line is green at −67 dBm or better, yellow down to −70 dBm and red below.

3.5 Software Design

The application is organised around three main views, each serving a different stage of Wi-Fi analysis; the navigation follows network inspection → time-based observation → spatial visualisation.

Tab 1 – Access Point List:

This first stage lists the networks detected by the phone; a top panel shows the connected network with its IP address and link speed, and every access point shows:

- *Identity:* the SSID (network name), the BSSID (MAC address of that access point) and the vendor name derived from it.
- *Signal:* the received level in dBm with a colour-coded strength icon and an estimated distance.
- *Radio details:* channel, centre frequency and frequency range, channel width (20–160 MHz) and Wi-Fi standard such as 802.11n or 802.11ac.
- *Security and view options:* the security type (WPA2, WPA3 or open); the list can be sorted, grouped by SSID or channel, and filtered [8].

What the Access Point List conveys:

At the current location it shows which networks are present, which is strongest and how strong the user's own network is; whether that network has several access points (BSSIDs with one SSID); and which channels are crowded, since many access points on one channel interfere. It is a snapshot of one location, not a floor-plan map.

Tab 2 – Time Graph:

The Time Graph plots the signal strength of each access point, between −100 and 0 dBm, against successive scans, with one coloured line per access point and a legend. At the default scan interval of 5 s the visible window covers about 21 scans (under two minutes), and up to 200 scans are kept. It is used to:

- *Check stability* at a fixed position before recording a heat-map reading, waiting for a steady trace.
- *Find the edge of coverage* by watching the signal fall while walking away from the router or behind obstacles.
- *Detect fluctuations* that justify repeating a measurement, and *compare access points* such as the router and the repeater over time.

The graph shows when the signal changes and the heat map where weak coverage occurs; a time graph alone cannot establish the boundary of a dead zone.

Tab 3 – Heatmap:

The Heatmap tab relates measurements to a floor plan. It contains a home screen with *Create new heatmap* and *Previous heatmaps*; an image picker for the floor plan; a grid-size choice (Small, Medium or Large); a survey screen with the floor plan, the grid, a live status line (network, RSSI and number of readings) and the *Back*, *Reset* and *Generate* buttons; a result screen with the colour heat map, its legend and *Edit*, *Save* and *Export*; and a list of saved surveys with network, date, readings and grid size. Colour transitions show where more measurements are needed.

3.5.1 Algorithm

Algorithm A – Survey and recording (HeatmapFragment):

1. Pick a floor-plan image (scaled to at most 1800 pixels) and a grid of 16, 12 or 8 columns; rows follow from the image ratio so squares stay square.
2. Tap a square to select it and again to record. Without a connection show *Connect to a Wi-Fi network first*; on a different network show *This survey is for <SSID>*.
3. Otherwise store the connected access point's RSSI (scan-averaged, refreshed every 5 s); tapping a recorded square twice deletes it.
4. With three or more readings enable *Generate*, then offer *Edit*, *Save* and *Export*.

Algorithm B – Heat-map generation (HeatmapRenderer and HeatmapIdw):

For every pixel of a small heat bitmap (about 200 pixels on the longer side), the RSSI is estimated from the readings within an influence radius R = 2.5 squares by inverse-distance weighting [6], as illustrated in Figure 3.7:

$$\mathrm{RSSI}(P) = \Sigma\, w_i \cdot \mathrm{RSSI}_i \,/\, \Sigma\, w_i\,, \quad w_i = 1 \,/\, d_i^2 \tag{3.3}$$

where d_i is the distance in squares to reading i. Opacity is 1 within 0.4R (one square) of the nearest reading and falls to zero at R, so areas without readings stay uncoloured. The hue is:

$$H = 120° \times \text{clamp}((\text{RSSI} + 90) / 45, 0, 1) \quad (3.4)$$

so −90 dBm is red, −67.5 dBm yellow and −45 dBm green; for example, −50 and −70 dBm at equal distances give −60 dBm.

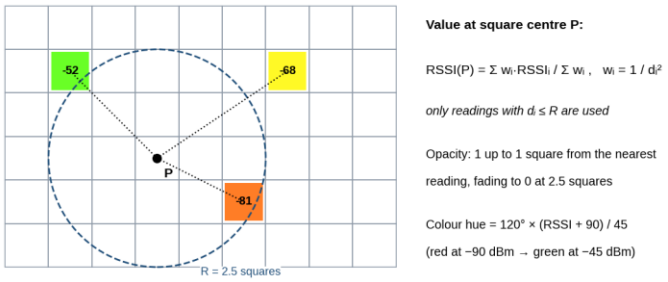


FIGURE 3.7 – INVERSE-DISTANCE-WEIGHTED INTERPOLATION USED FOR THE HEAT MAP

Algorithm C – Repeater principle and configuration (prebuilt firmware):

The ESP8266 repeater, which must sit where the router's signal is usable, is configured with prebuilt firmware rather than programmed from scratch:

1. Flash the prebuilt firmware binary over USB with the flashing utility, restart the board and check that its open configuration network *MyAP* appears.
2. Connect a phone or computer to it, open http://192.168.4.1 and enter the main network's SSID and password and the repeater's own network settings.
3. Save the settings; clients then receive 192.168.4.x addresses and their traffic is relayed through NAPT [5], which should be verified with a test device.

Operation with several access points of the same network:

- *Uplink choice:* With several access points sharing one SSID, the ESP8266 joins one of them (any by default, or a specific BSSID set in the firmware) [10].
- *Single radio:* The soft-AP uses the channel of the chosen access point; if the uplink is lost and the ESP8266 reconnects to another access point on a different channel, its clients briefly reconnect.

- *Separate network name:* Clients join the repeater's own SSID without seamless roaming; with several ESP8266 repeaters, the firmware's automesh mode lets all offer the original SSID and pick the best uplink.
- *In the application:* The Access Point List and Time Graph show each access point (BSSID) separately, while the heat map accepts readings from any access point with the survey's SSID, so it shows the combined coverage of the whole network as the phone roams.

3.5.2 Flowchart

The flowcharts of the survey procedure and of the repeater start-up are shown in Figure 3.8.

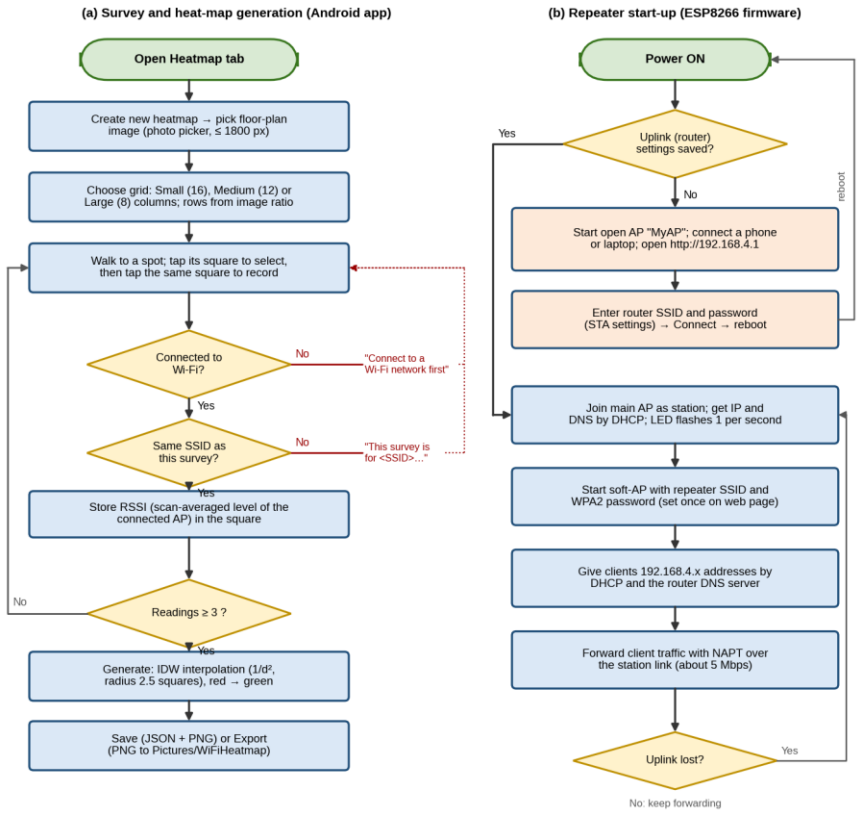


FIGURE 3.8 – FLOWCHARTS OF (A) THE HEAT-MAP SURVEY AND (B) THE REPEATER START-UP

Chapter 4

IMPLEMENTATION

4.1 Hardware Implementation Details

The repeater has only four main parts, but because it combines a lithium-ion cell with continuous radio operation it was assembled carefully into a compact unit.

4.1.1 Step-by-step hardware assembly process

1. *Inspection:* The cell voltage was checked with a multimeter and the parts inspected for damage.
2. *Holder wiring:* The red lead was soldered to B+ and the black lead to B− of the module, with heat-shrink sleeving.
3. *Polarity check:* The holder polarity was confirmed before inserting the cell, as a reversed cell damages the module.
4. *Firmware flashing:* With the NodeMCU on USB, a prebuilt repeater firmware binary obtained from an online repository was written with a flashing utility (Espressif's Flash Download Tool [13]).
5. *Laptop-powered configuration:* Powered from the laptop, the board was configured at 192.168.4.1, where the repeater SSID and network settings were changed.
6. *Portable assembly:* The cell, holder and booster module were assembled and connected to the NodeMCU through a micro-USB cable for further testing.

4.1.2 Challenges faced during hardware building

- *Power-bank auto shut-off:* Such modules switch off at very low load; in station + soft-AP mode the ESP8266 does not modem-sleep, so its 85–90 mA load keeps the output on.
- *Flashing the correct files:* Each binary must go to its own address with the right flash size, or the board keeps restarting.
- *Antenna, cable and battery:* The PCB antenna is kept clear of metal, a short USB cable avoids resets, and the holder avoids soldering to the cell.

- *Power-module verification:* The output voltage, current capacity, battery protection and charging behaviour of the module must be checked on the actual module; charging should not be assumed to be safe without verification.

4.2 Programming Details

4.2.1 Main program structure

Kotlin and Android (mapping application): The application is written in Kotlin, a statically typed language from JetBrains that runs on the Java virtual machine and is the preferred language for Android [14]. It is built with Gradle in Android Studio for Android 7.0 (API level 24) and later. The project repository adds a new package, wifi/heatmap, plus changes to the navigation (Heatmap is the third bottom tab after Access Points and Time Graph), the colour theme and the text resources [9]. Its files are listed in Table 4.1 and their relationships are shown in Figure 4.1.

TABLE 4.1 – FILES OF THE HEATMAP MODULE AND THEIR RESPONSIBILITIES

File	Lines	Responsibility
HeatmapModels.kt	54	Cell, Survey and GridSize types; gridRows(); MINIMUM_READINGS = 3
HeatmapIdw.kt	123	IDW interpolation, RSSI-to-colour mapping and heat-bitmap renderer
HeatmapCanvasView.kt	332	Draws floor plan, grid, readings, selection and legend; converts taps to squares
HeatmapFragment.kt	482	Screens, recording, generation, save, export and saved-survey list
SurveyStore.kt	113	Saves surveys as JSON and floor plans as PNG in private storage
HeatmapExporter.kt	61	Exports the heat map as PNG to Pictures/WiFiHeatmap
fragment_heatmap.xml	396	Layout of the screens and their buttons
HeatmapIdwTest.kt (test)	99	Seven unit tests of the interpolation and grid logic

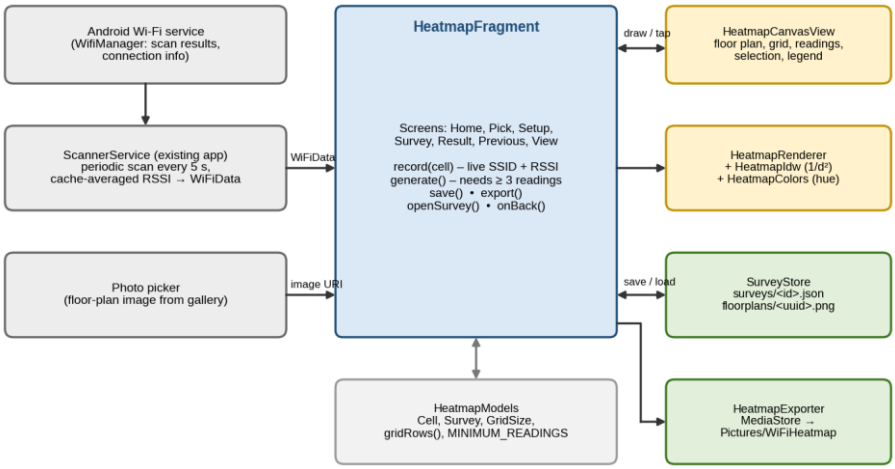


FIGURE 4.1 – SOFTWARE ARCHITECTURE OF THE HEATMAP MODULE

C firmware (repeater): The repeater runs pre-built open-source C firmware for the ESP8266 non-OS SDK with an lwIP stack extended for NAPT [10]; it needs no project code and is configured through its web page.

4.2.2 Important functions and their logic

- *record(cell):* Stores the RSSI only if the phone is connected and on the survey's network.
- *generate():* Needs at least three readings and calls the renderer with the grid size.
- *HeatmapIdw.interpolate() and HeatmapRenderer.render():* Compute the $1/d^2$ average and opacity for every pixel and colour it.
- *SurveyStore and HeatmapExporter:* Save surveys as JSON with a PNG floor plan, list and delete them, and export images through MediaStore.

4.2.3 Code logic

The core of the interpolation, taken from HeatmapIdw.kt, is:

```
fun interpolate(
    samples: List<Sample>,
    x: Float,
    y: Float,
    radius: Float = INFLUENCE_CELLS,
): Result? {
    val radiusSquared = radius * radius
    var weightSum = 0f
    var weightedSum = 0f
    var nearest = Float.MAX_VALUE
    for (sample in samples) {
        val dx = sample.x - x
        val dy = sample.y - y
        val distanceSquared = dx * dx + dy * dy
        if (distanceSquared > radiusSquared) continue
        if (distanceSquared < EXACT) return Result(sample.rssi.toFloat(), 1f)
        val weight = 1f / distanceSquared
        weightSum += weight
        weightedSum += weight * sample.rssi
        if (distanceSquared < nearest) nearest = distanceSquared
    }
    if (weightSum == 0f) return null
    val core = radius * CORE_FRACTION
    val distance = sqrt(nearest)
    val alpha = if (distance <= core) 1f else (1f - (distance - core) / (radius -
core)).coerceIn(0f, 1f)
    return Result(weightedSum / weightSum, alpha)
}
```

Readings beyond the radius are ignored and the others are weighted by $1/d^2$; with no reading in reach the function returns null and the area stays uncoloured. Similarly, record() refuses readings when disconnected or on another SSID.

4.3 Interfacing of Sensors, Actuators, Modules

4.3.1 Pin connections and interfacing diagram

Table 4.2 lists the connections; the interfacing diagram is Figure 3.2.

TABLE 4.2 – PIN CONNECTIONS OF THE REPEATER

From	To	Purpose
Holder + (red lead)	Module B+ pad	Battery positive
Holder − (black lead)	Module B− pad	Battery negative
Module USB-A (VBUS, GND)	NodeMCU micro-USB (cable)	5 V supply
NodeMCU VIN rail (5 V)	AMS1117 input (on board)	3.3 V for the ESP-12E
GPIO2 (D4)	On-board blue LED	Firmware status LED
GPIO1 / GPIO3 (TX / RX)	CP2102 bridge (on board)	Flashing and serial console

4.3.2 Communication protocols used

- *IEEE 802.11 b/g/n:* Uplink and extender access point at 2.4 GHz with WPA2-PSK [1].
- *DHCP, DNS and NAPT:* Clients get 192.168.4.x addresses and the main DNS server; NAPT routes their packets [5].
- *HTTP and console:* Configuration page at 192.168.4.1; console over UART (115200 baud) or Telnet port 7777 [10].

4.3.3 How each module communicates with the microcontroller

- *Access point and clients:* The station interface associates with the main access point; clients join the soft-AP and their traffic is translated and forwarded.
- *Smartphone:* The Heatmap module reads Wi-Fi data from the app's scanner service, fed by Android's WifiManager [15].
- *Laptop:* The Flash Download Tool writes firmware through the CP2102, and a browser configures the repeater.

4.4 Development Tools Used

4.4.1 List of software and hardware tools

- *Application:* Android platform and Wi-Fi scanning, Kotlin with Gradle in Android Studio, the open-source analyzer code base and GitHub for the repository.
- *Repeater:* Firmware flashing utility, prebuilt repeater firmware, CP2102 driver and a web browser for the page at 192.168.4.1.
- *Testing and hardware:* Android phone, laptop, floor-plan images and measurement notes, NodeMCU, 18650 cell and holder, booster module, micro-USB cable and multimeter.

4.4.2 Purpose of each tool

- *Android Studio and GitHub:* Building, testing and installing the application and keeping its source and history.
- *Flashing utility and browser:* Writing the prebuilt firmware and configuring the repeater.
- *Laptop, phone and multimeter:* Initial USB power, running the application and checking voltages and polarity.

Chapter 5

PERFORMANCE ANALYSIS

Note on the results in this chapter: Only results demonstrated by the prototype are reported as results. Coverage improvement, battery runtime and sustained operation have not yet been quantified; they require the before-and-after measurements described in Section 5.2. Reference images from other sources are used only to illustrate the expected form of a heat map and are not measurements of this project.

5.1 Testing Methodology

Testing verifies each component independently before the complete coverage-improvement workflow is evaluated.

5.1.1 Types of testing done

- *Unit testing:* The project includes seven automated JUnit tests (HeatmapIdwTest) for the interpolation and grid functions of the heat map.
- *Component testing:* The application (launch, access-point list, time graph, floor-plan upload and heat map) and the repeater (flashing, configuration, operation and battery power) are tested separately.
- *End-to-end testing:* The same positions are measured before and after placing the repeater to examine whether coverage changes.

5.1.2 Test cases

Table 5.1 lists the unit tests included in the project and Table 5.2 the test plan for the prototype, with what has to be recorded in each test.

TABLE 5.1 – UNIT TESTS OF THE HEATMAP MODULE (HEATMAPIDWTEST)

Test	Input	Expected result
exactSampleReturnsItsValueWithFull Alpha	−60 dBm at (0.5, 0.5); query the same point	RSSI −60, opacity 1
noSampleInsideRadiusReturnsNull	Reading at (0.5, 0.5); query (10, 10)	No value (uncoloured)
alphaFadesBetweenCoreAndRadius	Query 1.75 squares from a reading	Opacity 0.5
equidistantSamplesAreAveraged	−50 and −70 dBm, both 1 square away	RSSI −60
nearerSampleDominates	−50 dBm at 1 square, −90 dBm at 2 squares	RSSI above −60
fractionIsClampedToZeroAndOne	−120, −10 and −67.5 dBm	0, 1 and 0.5
gridRowsKeepsSquareCells	12 columns on 1200×1800; 8 on 800×100	18 rows; 1 row

TABLE 5.2 – TEST PLAN OF THE PROTOTYPE

Test	Procedure	What to record
Application launch	Open the app on the test phone	Whether the app starts successfully
Access-point scanning	Scan the nearby wireless environment	Networks displayed and visible details
Time graph	Select a network and observe its graph	Whether the graph updates and reflects changes
Floor-plan upload	Select a floor-plan image	Whether the image loads correctly
Measurement association	Record observations at selected floor-plan positions	Position labels and corresponding readings
Heat-map generation	Generate the heat map from the observations	Whether it appears and represents the recorded points
Firmware flashing	Flash the selected firmware binary	Whether the board starts the expected firmware
Configuration access	Join the configuration network and open 192.168.4.1	Whether the page loads and settings can be changed
Repeater operation	Configure the main network and test a connected device	Whether the device connects and traffic works
Battery-powered operation	Run the device from the battery power chain	Whether operation stays stable
Coverage comparison	Measure the same positions before and after placement	Actual readings and connectivity at each position

5.1.3 Testing conditions and environment

For a fair comparison, the phone, the measurement positions, the selected network and the test procedure are kept the same in every session, results are repeated when they fluctuate, and failures

are recorded as well as successes. The department floor plans prepared as survey maps are shown in Figure 5.1; the first survey is planned on the ENTC ground floor with the Medium grid of 12 × 11 squares.

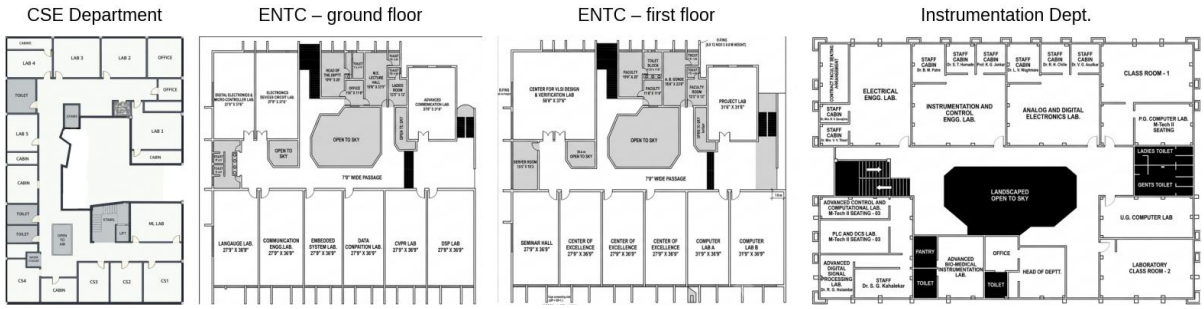


FIGURE 5.1 – DEPARTMENT FLOOR PLANS PREPARED AS SURVEY MAPS

5.2 Experimental Setup

5.2.1 Description of the testing setup

A. Application validation:

- Confirm that the access-point list shows the networks detected by the phone and that the time graph of a selected network can be observed.
- Verify that an uploaded floor plan and the associated measurements produce the expected heat map, and that it corresponds to the data actually collected.

B. Repeater validation:

- Confirm that the ESP8266 starts the flashed firmware, that the configuration page opens at 192.168.4.1 and that the repeater connects to the intended network.
- Connect a test device through the repeater and repeat the test with the battery-powered arrangement.

C. End-to-end validation:

1. Record the original network condition at the selected positions and identify the weak area on the heat map.
2. Place the repeater where the original network is usable, record the same positions again and compare signal and connectivity.

Figure 5.2 shows the planned survey grid, measurement checkpoints and proposed repeater position on the ENTC ground floor. A successful firmware flash or configuration-page login does not by itself prove that the repeater improves coverage; that claim requires the before-and-after test.

5.2.2 Photographs of experimental setup and working model

Figure 5.3 shows the NodeMCU used for the repeater, connected over USB for flashing and initial laptop-powered testing; the other component photographs are in Section 3.4.1 and Appendix E.

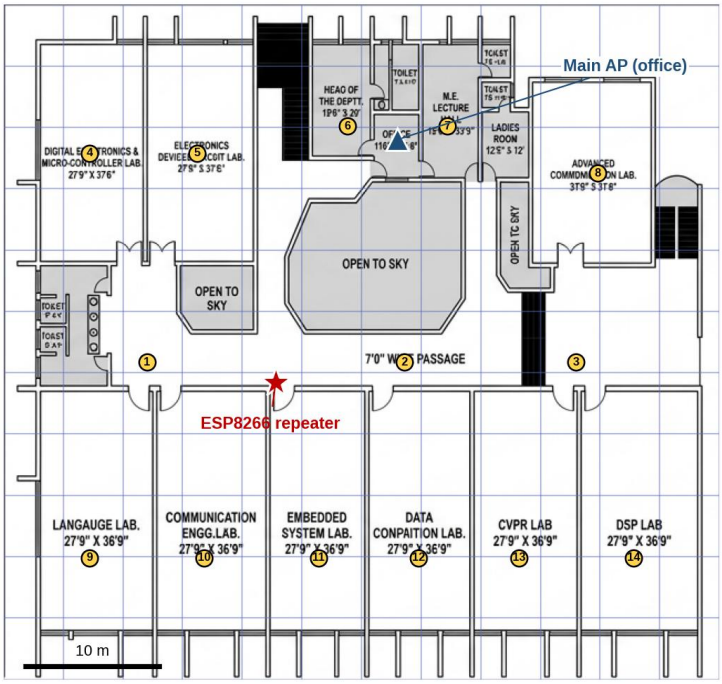


FIGURE 5.2 – PLANNED SURVEY GRID, CHECKPOINTS AND PROPOSED REPEATER POSITION

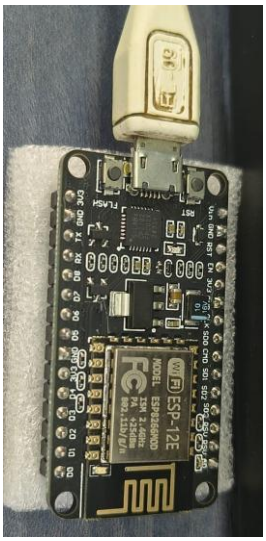


FIGURE 5.3 – NODEMCU CONNECTED OVER USB

5.3 Results

5.3.1 Measured results in tabular form

No coverage measurements have been recorded yet; Table 5.3 lists the results demonstrated by the prototype, and the before-and-after readings will go in Appendix F.

TABLE 5.3 – RESULTS DEMONSTRATED BY THE CURRENT PROTOTYPE

Result	Status
Android application based on an open-source Wi-Fi analysis code base	Built; source in the project repository
Access Point List for inspecting nearby networks	Demonstrated
Time Graph for observing signal changes over time	Available in the application
Floor-plan Heatmap module (upload, grid survey, IDW heat map, save, export)	Implemented in source; to be validated with real measurements
ESP8266 flashed with prebuilt repeater firmware	Completed
Web configuration at 192.168.4.1 (repeater SSID and settings changed)	Completed
Laptop-powered repeater prototype	Completed
Portable 18650 power arrangement	Assembled; operation to be tested

The following results still require experimental evidence: generation of a heat map from real measurements in the current build, reliable operation of the repeater from the battery, a measurable improvement in coverage after placement, and a comparison showing which rooms improved and which remained weak.

5.3.2 Graphs and charts

Measured heat maps will be added after the validation survey. The heat map uses the colour scale of Figure 5.4, taken from the application's source code: the hue moves from red at −90 dBm through yellow at −67.5 dBm to green at −45 dBm, so a dead zone (below −80 dBm) appears red to orange.

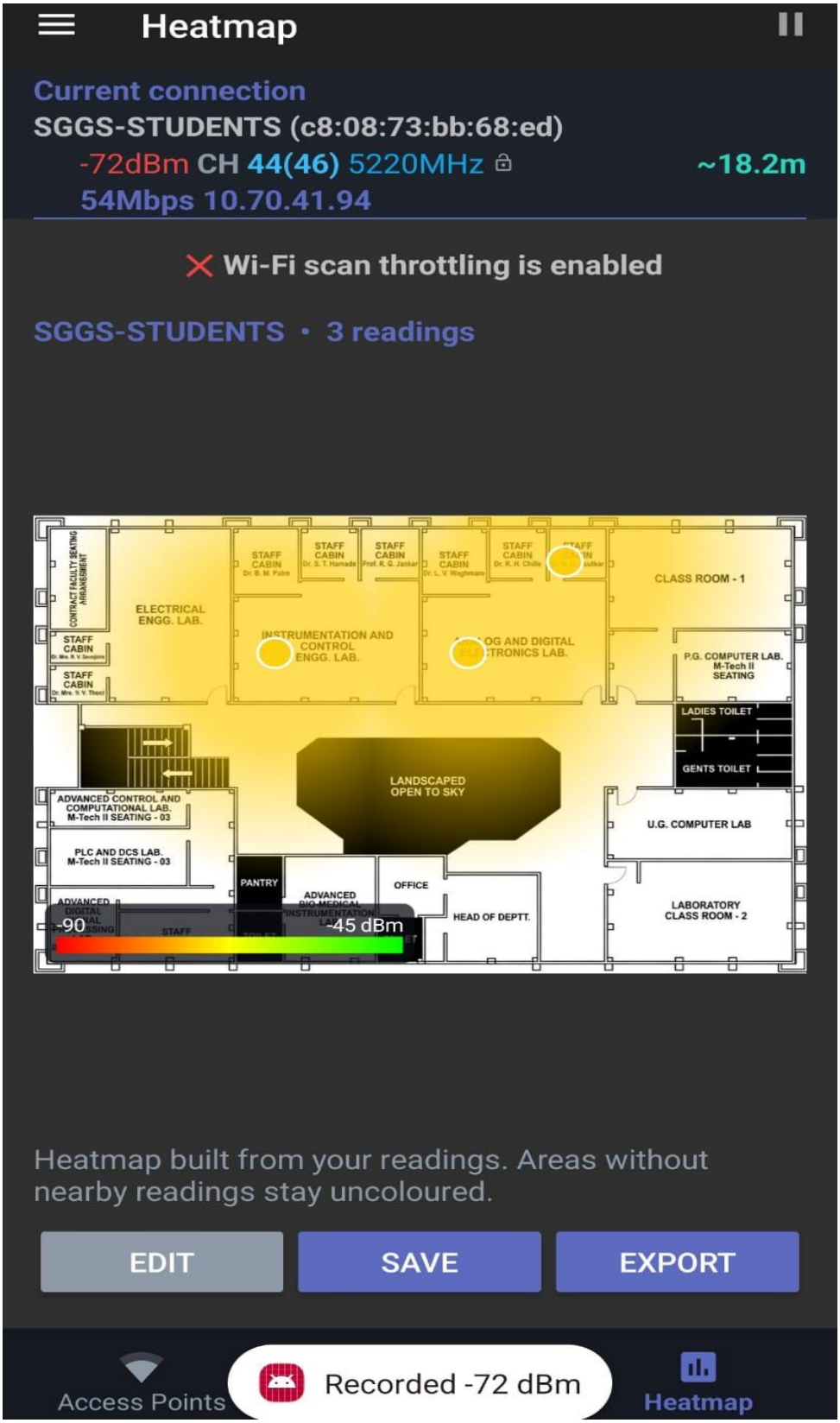


FIGURE 5.4 – COLOUR SCALE USED BY THE HEAT MAP OF THE APPLICATION

5.4 Performance Analysis and Comparison with Existing Systems

5.4.1 Analysis of obtained results

The prototype demonstrates the two halves of the workflow separately: the application inspects networks and observes them over time, with a unit-tested heat-map module in its source, and the ESP8266 was flashed, configured and powered from a laptop and then from the 18650 arrangement. The combined cycle of measuring, placing and measuring again, needed before any coverage improvement can be claimed, has not yet been shown.

5.4.2 Comparison table with existing systems

TABLE 5.4 – COMPARISON OF THE PROPOSED SYSTEM WITH EXISTING SYSTEMS

Feature	Range extender	Mesh Wi-Fi system	Proposed system
Approximate cost	₹1,200–₹3,000	₹8,000 and above	Under ₹1,000
Power source	Mains socket	Mains socket	18650 cell (runtime to be measured)
Dead-zone mapping	Not included	Basic (companion app)	Floor-plan heat map on the phone
Placement method	Trial and error	App-guided	Measured, then verified
Throughput after extension	Tens of Mbps	100 Mbps and above	Reduced; about 5 Mbps reported for the firmware [10]
Frequency bands	2.4 GHz or dual-band	Dual or tri-band	2.4 GHz only
Programmability	Closed firmware	Closed firmware	Open-source app and firmware

5.4.3 Performance parameters

- *Signal strength (RSSI):* At every checkpoint, on the main network and on the repeater network.
- *Connectivity:* Whether pages load and calls work at each checkpoint.
- *Throughput and latency:* Speed tests through the main network and the repeater (expected to be slower).
- *Battery runtime:* Operating time of the repeater from a full charge.

5.5 Discussion on Results

5.5.1 Interpretation of results

The work establishes a practical workflow rather than a final coverage figure: the application shows where the signal is weak, and the repeater can be deployed where the network is still usable. On the heat map, weak-coverage colours mark poorer reception and colour transitions show where more measurements are needed.

5.5.2 Success points and reasons

- *Firmware deployment and configuration* were achieved through the device-hosted web page, without custom firmware.
- *The three views and the portable power chain* separate the analysis stages and make the repeater movable.

5.5.3 Failures or unexpected behavior and reasons

- *Separate operations:* Heat-map analysis and repeater configuration have no automatic link.
- *Coverage not yet quantified:* Without a before-and-after measurement, an improvement cannot yet be claimed.
- *Unverified power module:* Its protection and charging behaviour still have to be checked.

5.6 Limitations

- *Incomplete validation:* The build may not include every feature of the intended end-to-end workflow.
- *No automatic placement, not yet quantified:* The position is chosen manually, and runtime and coverage gain are still to be measured.

5.7 Advantages

- *Low cost and portable:* Under ₹1,000 (approximately) and needs no power socket.
- *Measurement-driven and flexible:* Weak areas are measured before hardware is placed, on any floor-plan image.

Chapter 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The project combines Wi-Fi analysis with a practical hardware-based approach to extending coverage. The Android application, built on an open-source Wi-Fi analysis code base, provides a way to inspect networks through the Access Point List, observe signal changes through the Time Graph and relate measurements to a floor plan through the Heatmap module. The ESP8266 repeater, flashed with prebuilt firmware and configured through its web page at 192.168.4.1, offers a portable, physical method for testing coverage extension.

The prototype has demonstrated network inspection, firmware deployment and configuration, laptop-powered operation and the assembly of a battery-powered arrangement, and it has established the measurement-driven workflow of measuring first, mapping the weak areas, placing the repeater strategically and verifying the result. The next step is to validate the complete workflow through repeatable measurements and a documented before-and-after comparison.

6.2 Future Scope

- *Complete the heat-map workflow:* Finalise floor-plan upload and measurement placement, with a readable legend and saved, reopenable sessions.
- *Measurement quality and comparison:* Repeat readings per location and compare a baseline survey with a second one after configuration.
- *Placement guidance:* Suggest positions that balance the link to the router with coverage of the weak region.
- *Better portable hardware:* Verify the power module, measure battery runtime and build a safer, compact enclosure.
- *Integrate application and hardware:* Explore a guided workflow that connects survey, placement and verification.

REFERENCES

[1] Espressif Systems, “ESP8266EX Datasheet.” [Online]. Available: https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf

[2] VREM Software Development, “WiFiAnalyzer: Android application to analyze Wi-Fi signals,” GitHub repository. [Online]. Available: https://github.com/VREMSoftwareDevelopment/WiFiAnalyzer

[3] Vardhit-08, “WIFI_deadzone: Wi-Fi dead-zone mapper (Android application with Heatmap module),” GitHub repository. [Online]. Available: https://github.com/Vardhit-08/WIFI_deadzone

[4] M. Ger, “esp_wifi_repeater: A full functional WiFi NAT router for the ESP8266,” GitHub repository. [Online]. Available: https://github.com/martin-ger/esp_wifi_repeater

[5] ESP8266 Community, “ESP8266 Arduino Core Documentation.” [Online]. Available: https://arduino-esp8266.readthedocs.io/

[6] Espressif Systems, “Flash Download Tools,” support downloads. [Online]. Available: https://www.espressif.com/en/support/download/other-tools

[7] JetBrains, “Kotlin Documentation.” [Online]. Available: https://kotlinlang.org/docs/home.html

[8] Android Developers, “WifiManager – API reference,” Google. [Online]. Available: https://developer.android.com/reference/android/net/wifi/WifiManager

APPENDIX

Appendix A: Bill of Materials

Table A.1 lists the components of the repeater with their quantities and costs.

TABLE A.1 – BILL OF MATERIALS OF THE REPEATER

S. No.	Component	Specification	Qty	Purpose	Cost (₹)
1	ESP8266 NodeMCU	ESP-12E module, CP2102 USB–UART	1	Repeater controller	≈ 280
2	18650 Li-ion cell	3.7 V, 2000 mAh, 7.4 Wh	1	Energy storage	365
3	2-in-1 power-bank module	T6845-C, 1 A charge, 5 V / 1 A out	1	Charging, protection, boost	175
4	18650 battery holder	Single cell, with wire leads	1	Holding the cell	≈ 25
5	Micro-USB cable	USB-A to micro-USB, short	1	5 V supply to NodeMCU	≈ 50
6	Computer / laptop	Existing	1	Flashing and initial power	—
				Total (approximate)	**≈ 895**

Costs of items 2 and 3 are actual purchase prices; values marked ≈ are approximate market prices.

Appendix B: Datasheets of major components

1. *ESP8266EX Wi-Fi SoC (Espressif Systems):* https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf
2. *T6845-C power-bank module:* https://grobotronics.com/powerbank-module-t6845-c.html
3. *NAT repeater firmware (configuration, LED codes, commands):* https://github.com/martin-ger/esp_wifi_repeater
4. *Espressif Flash Download Tools:* https://www.espressif.com/en/support/download/other-tools
5. *18650 Li-ion cell:* ratings printed on the cell (3.7 V, 2000 mAh, 7.4 Wh, IS 16046 / IEC 62133); charging practice as in [16].

Appendix C: Complete Source Code

The complete source code of the mapping application, including the Heatmap module, is available in the project repository [9]; it is based on an open-source Android Wi-Fi analyzer [8] licensed under GPLv3. The two core files of the Heatmap module are listed below (licence headers omitted). The repeater uses pre-built open-source firmware [10] and contains no project code.

```
// ---------------- HeatmapModels.kt ----------------
package com.vrem.wifianalyzer.wifi.heatmap

import androidx.annotation.StringRes
import com.vrem.wifianalyzer.R
import kotlin.math.max
import kotlin.math.roundToInt

const val MINIMUM_READINGS = 3

enum class GridSize(
    @get:StringRes @param:StringRes val textResource: Int,
    val columns: Int,
) {
    SMALL(R.string.heatmap_grid_small, 16),
    MEDIUM(R.string.heatmap_grid_medium, 12),
    LARGE(R.string.heatmap_grid_large, 8),
}

data class Cell(
    val column: Int,
    val row: Int,
)

data class Survey(
    val id: String,
    val ssid: String,
    val timestamp: Long,
    val image: String,
    val columns: Int,
    val readings: Map<Cell, Int>,
)

fun gridRows(
    columns: Int,
    imageWidth: Int,
    imageHeight: Int,
): Int = max(1, (columns.toFloat() * imageHeight / imageWidth).roundToInt())

// ---------------- HeatmapIdw.kt ----------------
package com.vrem.wifianalyzer.wifi.heatmap

import android.graphics.Bitmap
import android.graphics.Color
import kotlin.math.max
import kotlin.math.sqrt

/**
 * Inverse distance weighting (power 2) over the readings, limited to a fixed influence
radius.
 * All coordinates are in grid-cell units, a reading sits at the centre of its cell.
```

```
 * Where no reading is within the radius the result is null, so that area stays neutral (no
invented data).
 */
object HeatmapIdw {
    const val RSSI_WEAK = -90
    const val RSSI_STRONG = -45
    const val INFLUENCE_CELLS = 2.5f
    const val CORE_FRACTION = 0.4f
    private const val EXACT = 0.0001f

    class Sample(
        val x: Float,
        val y: Float,
        val rssi: Int,
    )

    class Result(
        val rssi: Float,
        val alpha: Float,
    )

    fun fraction(rssi: Float): Float = ((rssi - RSSI_WEAK) / (RSSI_STRONG -
RSSI_WEAK).toFloat()).coerceIn(0f, 1f)

    fun interpolate(
        samples: List<Sample>,
        x: Float,
        y: Float,
        radius: Float = INFLUENCE_CELLS,
    ): Result? {
        val radiusSquared = radius * radius
        var weightSum = 0f
        var weightedSum = 0f
        var nearest = Float.MAX_VALUE
        for (sample in samples) {
            val dx = sample.x - x
            val dy = sample.y - y
            val distanceSquared = dx * dx + dy * dy
            if (distanceSquared > radiusSquared) continue
            if (distanceSquared < EXACT) return Result(sample.rssi.toFloat(), 1f)
            val weight = 1f / distanceSquared
            weightSum += weight
            weightedSum += weight * sample.rssi
            if (distanceSquared < nearest) nearest = distanceSquared
        }
        if (weightSum == 0f) return null
        val core = radius * CORE_FRACTION
        val distance = sqrt(nearest)
        val alpha = if (distance <= core) 1f else (1f - (distance - core) / (radius -
core)).coerceIn(0f, 1f)
        return Result(weightedSum / weightSum, alpha)
    }
}

object HeatmapColors {
    private const val HUE_GREEN = 120f
    private const val HSV_SIZE = 3

    fun color(
        rssi: Float,
        alpha: Int,
        hsv: FloatArray = FloatArray(HSV_SIZE),
    ): Int {
        hsv[0] = HeatmapIdw.fraction(rssi) * HUE_GREEN
        hsv[1] = 1f
        hsv[2] = 1f
        return Color.HSVToColor(alpha, hsv)
```

```
    }
}

object HeatmapRenderer {
    private const val RENDER_MAX_SIDE = 200
    private const val MIN_SCALE = 4
    private const val HEAT_ALPHA = 170
    private const val HSV_SIZE = 3

    fun render(
        readings: Map<Cell, Int>,
        columns: Int,
        rows: Int,
    ): Bitmap? {
        if (readings.isEmpty() || columns < 1 || rows < 1) return null
        val scale = (RENDER_MAX_SIDE / max(columns, rows)).coerceAtLeast(MIN_SCALE)
        val width = columns * scale
        val height = rows * scale
        val samples = readings.map { (cell, rssi) -> HeatmapIdw.Sample(cell.column + 0.5f,
cell.row + 0.5f, rssi) }
        val hsv = FloatArray(HSV SIZE)
        val pixels = IntArray(width * height)
        for (j in 0 until height) {
            val y = (j + 0.5f) / scale
            for (i in 0 until width) {
                val x = (i + 0.5f) / scale
                val result = HeatmapIdw.interpolate(samples, x, y) ?: continue
                pixels[j * width + i] = HeatmapColors.color(result.rssi, (result.alpha *
HEAT_ALPHA).toInt(), hsv)
            }
        }
        return Bitmap.createBitmap(pixels, width, height, Bitmap.Config.ARGB_8888)
    }
}
```

Appendix D: User Manual / Operating Instructions

1. *Flashing (once):* Connect the NodeMCU over USB and flash the prebuilt firmware binary with the flashing utility.
2. *Configuring:* Restart the board, join MyAP, open http://192.168.4.1, enter the main network's SSID and password and the repeater's own settings, and save.
3. *Powering:* Insert a charged 18650 cell and connect the module's USB output to the NodeMCU.
4. *Inspecting networks:* Use the Access Point List and the Time Graph of a selected network.
5. *Surveying:* In the Heatmap tab choose *Create new heatmap*, a floor plan and a grid; tap a square twice to record; after three readings tap *Generate*, then *Save* or *Export*.
6. *Placing and verifying:* Place the repeater where the main network is usable near the weak area and repeat the survey.

Appendix E: Additional Photographs

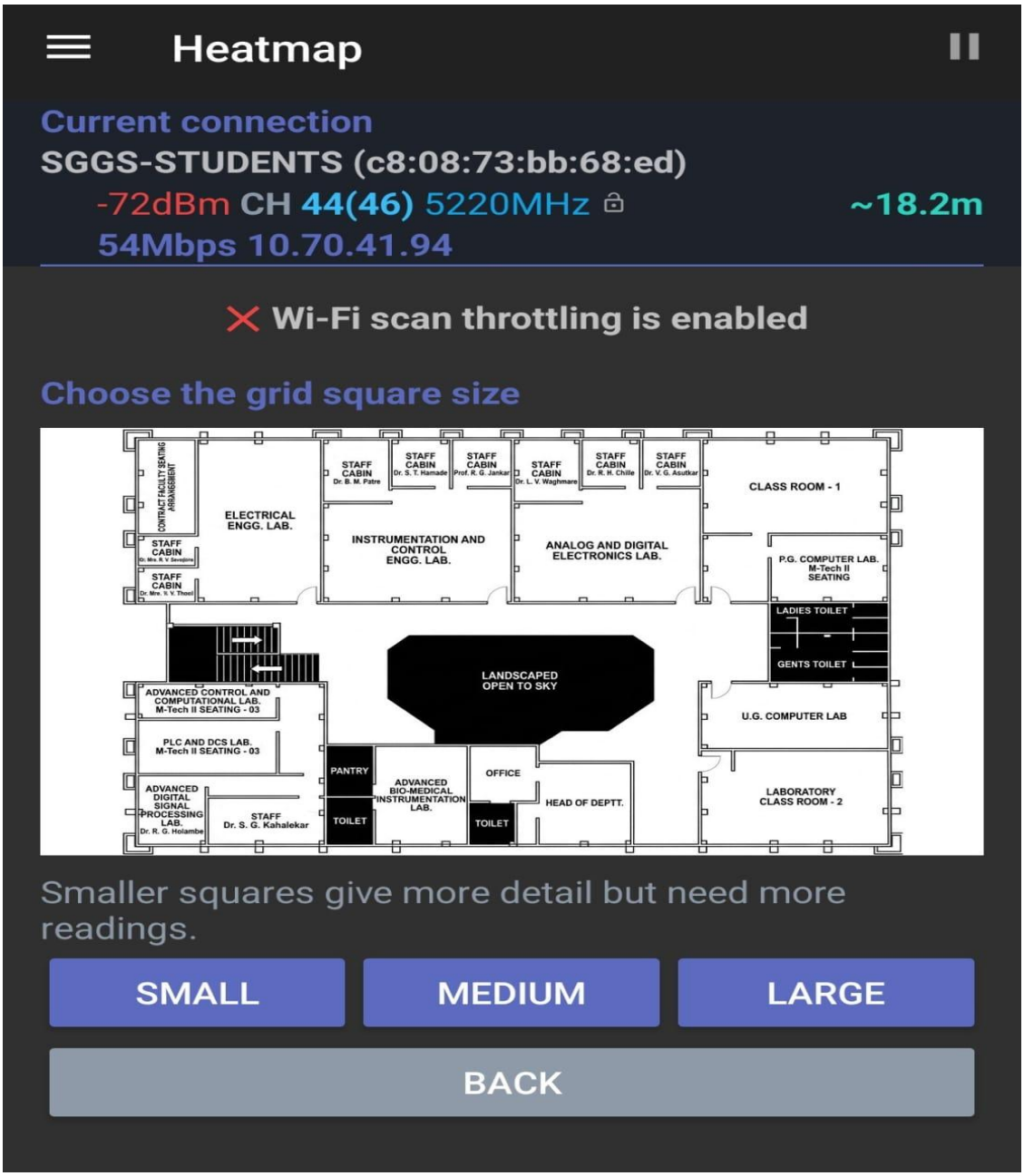




FIGURE E.1 – UNDERSIDE OF THE POWER-BANK MODULE (CONTROLLER IC AND BOOST INDUCTOR)



FIGURE E.2 – TYPICAL SINGLE-CELL 18650 BOOST ARRANGEMENT (REFERENCE IMAGE: LEAP#501)